CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

अमृतं तु विद्या

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 1: Array Operations: Traversal, Rotation, and Frequency Analysis

Practical Aim:- To implement fundamental array operations.

Problem 1:- A bakery prepares n items every morning and places them in a display row. At the end of each

hour, the leftmost item is moved to the rightmost position to make room for fresh stock at the

front. Given the initial row and the number of hours h, print the final display order.

Describe the approach you used. How does it behave when h is very large, say 10 million? Is

there a way to get the correct result without performing the operation h times?

Practical Aim:- To implement fundamental array operations.

For Code:-

https://github.com/Brijesh2212/FDSA_A_SHP/tree/main/PRAC-1

Output:

```
PS D:\dsa practicals> cd "d:\dsa practicals\" ; if ($?) { g++ prac1.cpp -o prac1 } ; if ($?) { .\prac1 }
5
3
1
3
4
5
6
1 3 4 5 3
PS D:\dsa practicals> cd "d:\dsa practicals\" ; if ($?) { g++ prac1.cpp -o prac1 } ; if ($?) { .\prac1 }
3
1
1
5
6
1 1 5
PS D:\dsa practicals>
```

Problem Faced: To understand the concept of left rotation at the beginning

Outcome/Conclusion:

The program performs a left rotation of the array efficiently by using h % n. This avoids performing the rotation h times and gives the correct result in O(n) time, even when the number of rotations is very large.

Answer:
The solution would need to remove or ignore punctuation marks such as commas, periods, exclamation marks, and question marks before calculating the length of each word. This should be handled during the word-processing or input-processing step, before finding the length. For example, hello, would first be treated as hello, so its length would be calculated as 5 instead of 6.

Problem 2:- A library issues books to students and records each book's ID every time it is borrowed. At the

end of the month, the librarian wants to find all books that were borrowed more than once, as

those need priority restocking. Given the borrowing log, print all such book IDs.

Describe the approach you used. How many times does your solution scan the data? If the

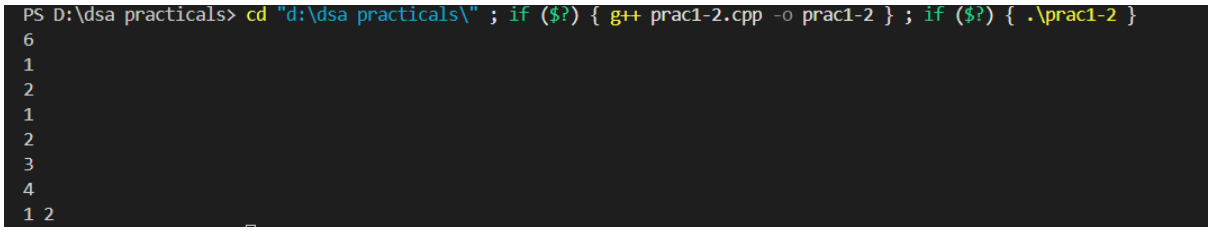
library had 100,000 borrow records, would your approach still be practical? Can you think of a

way that requires fewer passes?

For Code:-

https://github.com/Brijesh2212/FDSA_A_SHP/tree/main/PRAC-1

Output:



Problem Faced: The numbers which were repeating were printed the number of times they were repeated

Outcome/Conclusion:
The program uses nested loops to compare the elements of the array and identify duplicate values. It provides a simple way to find duplicates without requiring complex data structures.

Problem 3:- A school newspaper editor wants to highlight the most impressive word from a submitted article

on the front page. The word is chosen simply by length — the longest one wins. Given a

sentence, print the winning word and how many letters it has.

Describe the approach you used. What does your solution do when two words have the same

length? Is that behavior intentional, and can you think of a way to handle it explicitly?

For Code:-

https://github.com/Brijesh2212/FDSA_A_SHP/tree/main/PRAC-1

Output:

```
PS D:\dsa practicals> cd "d:\dsa practicals\" ; if ($?) { g++ prac1-3.cpp -o prac1-3 } ; if ($?) { .\prac1-3 }
Enter a sentence: hello i am brijesh
Longest word: brijesh
Number of characters: 7
PS D:\dsa practicals>
```

Problem Faced:- I was not able to figure out the logic for calculation of letters

Outcome/Conclusion:

The program finds the longest word in a given sentence by comparing the length of each word. If two or more words have the same maximum length, the solution keeps the first longest word encountered. This behavior is intentional and provides a simple way to handle ties. The tie can also be handled explicitly by storing and displaying all words having the maximum length.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] The display order repeats itself after a certain number of hours regardless of how large h gets. Without trying any specific example, explain why this repetition is inevitable and what determines when it first repeats.

Answer:

The display order repeats because there are only a fixed number of items in the row. Each hour, the leftmost item moves to the rightmost position, which is a left rotation. After n rotations, where n is the number of items, every item returns to its original position. Therefore, the display order repeats every n hours. The first repetition occurs after n rotations, and for a large value of h, we can use h % n to find the required effective rotations.

2. [Problem 2] If the same book ID appears three times in the log, should it appear once or three times in your output? Does your solution handle this correctly, and how would you verify it?

Answer:
If the same book ID appears three times, it should appear only once in the output because the purpose is to find duplicate book IDs, not to print every repeated occurrence. The solution handles this by checking whether the duplicate has already been processed or printed. To verify it, we can test the program with an input where one book ID appears three times and check that it is printed only once.

3. [Problem 3] The editor later decides that punctuation attached to a word (e.g. “hello,” or “great!”) should not count toward its length. How would this change your solution, and at which step would you handle it?